Build a Progressive Web App named **CraftShield** for custom jewellery orders.

CraftShield connects **Clients** who want custom jewellery with verified **Artisans** who design and manufacture jewellery. The purpose is to protect artisans from customers who cancel after work begins, refuse payment, or misuse an artisan's original jewellery design by getting a cheaper copy made elsewhere.

The application must have three separate user roles and separate user interfaces:

1. Client
2. Artisan
3. Admin

Use a responsive mobile-first design because this is a PWA. The app should be installable, work well on mobile devices, and support basic offline access for previously loaded pages.

Technology Stack

- Frontend: React
- Styling: Tailwind CSS
- Backend: Python FastAPI
- Database: MongoDB
- PWA: React PWA configuration, manifest, service worker
- Authentication: JWT with role-based access control
- AI: Voice assistant and customer churn-risk prediction
- Blockchain: Design ownership timestamp and verification record
- Payment: Payment gateway integration architecture, initially using mock payment flow for the prototype

Core Business Flow

1. A Client registers and logs in.
2. An Artisan registers and is verified by the Admin.
3. The Client browses artisan profiles and jewellery listings, or submits a custom jewellery request.
4. The Artisan reviews the request and sends a quotation, estimated timeline, and design proposal.
5. The Client accepts the quotation.
6. The Client pays a mandatory advance amount.
7. The advance payment is held securely by the payment workflow until the artisan accepts the order and begins production.
8. The Artisan can use the approved advance amount for material preparation according to the agreed order terms.
9. Before sharing a detailed jewellery design, sketch, or 3D CAD file with the Client, the system creates a blockchain ownership record for the design.
10. The Artisan updates production stages, and the Client tracks the order.
11. The Client pays the remaining amount before delivery or according to the agreed contract.
12. Both parties can review each other after completion.
13. The Admin can monitor users, orders, payments, complaints, and suspicious activity.

User Roles and Separate UI

Client UI

Create a Client dashboard with:

- Browse jewellery listings
- Search artisans by jewellery type, rating, price range, and location
- View artisan profiles and reviews
- Submit custom jewellery requests
- Upload reference images
- View quotations
- Accept or reject quotations
- Pay advance amount and final amount
- Track order status
- View shared designs
- Chat with artisan
- Raise a complaint or dispute
- View order history
- Voice assistant access

Artisan UI

Create an Artisan dashboard with:

- Profile and verification status
- Add, edit, and manage jewellery listings
- Upload product images
- Receive custom jewellery requests
- Send quotations and delivery estimates
- Accept or reject requests
- Upload design sketches or 3D CAD files
- Create blockchain ownership proof before sharing a design
- Update production progress
- View advance-payment status
- View earnings and order history
- Chat with clients
- View client churn-risk indicator only as a support signal, not as an automatic rejection decision
- Voice assistant access

Do not allow public users to register as Admin. Seed one prototype admin account:

- Username: `admin`
- Password: `1234`

Hash the password before storing it. Keep these credentials only for the local prototype and never expose them in a public production deployment.

Authentication and Authorization

Implement:

- Separate login pages or role selection for Client and Artisan
- Admin-only login access
- JWT authentication
- Protected routes
- Role-based API authorization
- Password hashing
- Logout
- Redirect users to their own dashboard after login

Rules:

- Clients can access only their own requests, orders, payments, chats, and reviews.
- Artisans can access only their own products, received requests, orders, designs, and payment information.
- Admins can access all records.
- A Client must not access Artisan dashboard routes.
- An Artisan must not access Client dashboard routes.

Jewellery Order Features

Support jewellery categories such as:

- Rings
- Necklaces
- Earrings
- Bracelets
- Bangles
- Chains
- Pendants
- Bridal Jewellery
- Custom Jewellery

A custom request must contain:

- Jewellery type
- Description
- Material preference
- Stone preference
- Quantity
- Budget
- Expected delivery date
- Reference image
- Preferred artisan, if selected

Order statuses:

```
Request Submitted
Quotation Sent
Quotation Accepted
Advance Payment Pending
Advance Payment Secured
Design in Progress
Production Started
Work in Progress
Quality Check
Ready for Delivery
Final Payment Pending
Delivered
Completed
Cancelled
Disputed
```

Payment Protection

Design the payment module as an escrow-style workflow.

For the Sprint 1 prototype:

- Use mock payment states rather than real banking transactions.
- Record advance amount, final amount, payment status, transaction reference, and timestamps.
- Do not claim that the application directly locks money in a bank account.
- Structure the system so that a future payment gateway such as Razorpay can hold, authorize, release, refund, or transfer funds according to the platform rules and applicable regulations.

Payment rules:

- Production cannot start until advance payment is confirmed.
- The advance amount should be visible to both Client and Artisan.
- Refund or release decisions must be handled through platform rules and Admin dispute review.
- Keep complete payment history for every order.

AI Features

1. Multilingual Voice Assistant

Add an AI voice assistant designed mainly for accessibility.

Initial supported languages:

- English
- Tamil
- Telugu
- Malayalam
- Kannada

The assistant should help users:

- Understand how to place an order
- Find jewellery categories
- Check order status
- Understand payment status
- Navigate the app
- Explain common platform rules

Use speech-to-text and text-to-speech architecture. For the first prototype, implement the UI, language selector, voice input button, and mock assistant responses if a full AI service is not available.

2. Customer Churn-Risk Prediction

Build a churn-risk or cancellation-risk feature for Clients using historical platform behaviour.

Possible input signals:

- Number of previous cancellations
- Number of abandoned custom requests
- Number of unpaid quotations
- Delay in accepting quotations
- Delay in making advance payments
- Number of disputes
- Refund requests
- Low review score from artisans
- Long inactivity period
- Repeated changes to order requirements
- High number of requests compared with completed orders

The churn score must not automatically block a Client. It should be used as a risk indicator for Artisans and Admins.

Use simple, explainable rules for the first version:

```
Low Risk: consistent completed orders and timely payments
Medium Risk: some delayed payments, abandoned requests, or frequent changes
```

```
High Risk: repeated cancellations, disputes, unpaid quotations, or payment
delays
```

Future versions can train a machine-learning model using anonymized historical order data.

3. Reliability and Fraud-Prevention Rules

Add a reliability score for both Clients and Artisans.

Client reliability factors:

- Completed order percentage
- On-time payment percentage
- Cancellation rate
- Dispute rate
- Review score
- Account verification status

Artisan reliability factors:

- On-time delivery percentage
- Order completion rate
- Client review score
- Verified identity status
- Dispute rate
- Response time to requests

Show these scores transparently, with an explanation of why the score exists. Do not make automated decisions that permanently deny users access without Admin review.

Blockchain Design Ownership Module

Create a design ownership record module.

When an Artisan uploads a jewellery sketch, design image, or 3D CAD file:

1. Generate a SHA-256 hash of the original file.
2. Store the file securely in application storage.
3. Store the hash, Artisan ID, Order ID, upload timestamp, file name, and file version in MongoDB.
4. Create a blockchain transaction or a mock blockchain transaction for Sprint 1.
5. Store the blockchain transaction ID and verification status.
6. Allow the Artisan and Admin to verify whether an uploaded file matches the stored hash.

Important: the blockchain record proves that a specific file hash was recorded at a specific time. It supports evidence of prior possession and timestamped ownership claims, but it does not automatically guarantee legal copyright ownership or make every dispute court-admissible. Legal validity depends on jurisdiction, evidence, contracts, and applicable intellectual-property law.

Main Pages

```
/
 /login
 /register
 /client/dashboard
 /client/artisans
 /client/products
 /client/custom-request
 /client/orders
 /client/payments
 /client/chat
 /client/profile
 /artisan/dashboard
 /artisan/products
 /artisan/add-product
 /artisan/requests
 /artisan/orders
 /artisan/designs
 /artisan/payments
 /artisan/chat
 /artisan/profile
 /admin/login
 /admin/dashboard
 /admin/users
 /admin/artisan-verification
 /admin/orders
 /admin/payments
 /admin/disputes
 /admin/blockchain-records
 /admin/analytics
```

Database Collections

```
users
artisan_profiles
products
custom_requests
quotations
orders
payments
design_records
blockchain_transactions
reviews
chats
```

```
messages
disputes
notifications
churn_risk_scores
```

UI Design Requirements

Create a modern, premium jewellery-inspired interface.

Use:

- Dark navy, gold, cream, and teal accents
- Clean cards and dashboard statistics
- Jewellery product cards
- Separate sidebar navigation for Client, Artisan, and Admin
- Status badges
- Tables for Admin management
- Responsive mobile navigation
- Loading, empty, success, and error states
- Accessibility-friendly labels and readable typography
- Voice assistant floating button

Development Scope

This is a student project. Build the application in modular phases.

Sprint 1 should focus on:

- Project setup
- PWA setup
- Separate role-based authentication
- Separate dashboard UI for Client, Artisan, and Admin
- Jewellery product listings
- Custom-order request flow
- Basic quotation and order-status flow
- MongoDB connection
- Mock payment states
- Mock blockchain design-record flow
- Voice assistant UI and basic multilingual mock responses
- Initial churn-risk rule engine

Do not implement real banking, real blockchain deployment, or a fully trained machine-learning model in Sprint 1. Keep the code modular so these can be integrated later.